

SourceWise Study Agent

Phase 3 — Mobile App Development Guide

This document covers the full specification for building the SourceWise mobile application using React Native and Expo. It reuses 100% of the existing backend APIs from Phase 2, adapts all UI screens for native mobile interactions (touch gestures, swipe cards, haptic feedback), adds push notification reminders, offline caching, and guides the complete build and app store submission process for both Android and iOS.

1. Mobile Tech Stack

React Native with Expo is the ideal choice because the entire backend and core business logic from Phases 1 and 2 is already complete. Expo abstracts native build complexity and provides a rich SDK for device features (camera, notifications, file system) without writing Objective-C or Kotlin.

Library / Tool	Mobile Purpose
Expo SDK 51+	Managed workflow. Provides access to device APIs without ejecting.
React Native 0.74+	Core framework. Renders native components on iOS and Android.
Expo Router v3	File-based routing (like Next.js) for React Native. Tab + Stack navigation.
React Native Paper	Material Design 3 components: Cards, Chips, FAB, ProgressBar, Dialogs.
Zustand	Same store slices as web — reused directly. Persist middleware for AsyncStorage.
Axios	Same axiosInstance reused. VITE_ prefix replaced with EXPO_PUBLIC_ in .env.
React Query	Same queries/mutations reused. Automatic background refetch on app focus.
Expo Notifications	Push notifications via Expo Push Service. Schedule local reminder alerts.
Expo FileSystem	Download and cache MP3 audio files for offline playback.
Expo Document Picker	Pick PDF/DOCX files from device storage or cloud (Google Drive, iCloud).
React Native Reanimated 3	60fps swipe animations for flashcards, gesture-driven interactions.
React Native Gesture Handler	Swipe left/right on flashcards, pull-to-refresh, bottom sheet drag.
AsyncStorage	Persist auth tokens, cached plans, offline flashcards between app sessions.
Victory Native XL	Mobile-optimized charts: radar (topic coverage), bar (study hours).
Expo Audio	Audio playback for revision MP3s. Background audio with lock screen controls.
EAS Build	Expo Application Services for cloud builds. Generates APK and IPA files.

2. Expo Project Structure

```
sourcewise-mobile/ app/ # Expo Router file-based routes (auth)/ # Public routes (login,
login.jsx signup.jsx (tabs)/ # Bottom tab navigator dashboard.jsx sources.jsx
planner.jsx profile.jsx summary/[sourceId].jsx flashcards/index.jsx quiz/index.jsx
audio/index.jsx _layout.jsx # Root layout with auth guard components/ # Reusable native
components FlipCard.jsx # 3D swipe card (Reanimated) AudioPlayerBar.jsx # Persistent
bottom audio bar ReadinessRing.jsx # SVG progress ring StudyDayCard.jsx # Planner day
block hooks/ # Shared with web (useAuth, useSources) store/ # Same Zustand slices +
AsyncStorage persist lib/ # axiosInstance (same as web) assets/ # App icon, splash
screen, fonts app.json # Expo config: name, bundleId, permissions eas.json # EAS build
profiles (preview, production)
```

3. Mobile Screen Specifications

Step 3.1 — Auth Screens (Login / Signup)

- Single scrollable column layout. Logo at top, form in middle, action button fixed at bottom.
- Keyboard-aware scroll view — form scrolls up when keyboard appears on small screens.
- Biometric login option (Expo LocalAuthentication): Face ID / fingerprint for returning users.
- Sign in with Google via Expo AuthSession (OAuth). Token returned to app as deep link callback.
- Form validation with React Hook Form — inline error messages below each field.
- Persist auth tokens in SecureStore (encrypted) not AsyncStorage.

Step 3.2 — Source Upload Screen

- Expo DocumentPicker for local file selection. Expo WebBrowser to let users import from Drive/Dropbox.
- Upload progress shown as animated linear progress bar with percentage label.
- Uploaded sources listed as swipeable row cards — swipe left reveals delete button.
- Pull-to-refresh on source list calls GET /sources to sync latest processing status.
- Haptic feedback (Expo Haptics) on upload complete and on delete confirmation.
- Empty state illustration with 'Upload your first document' call-to-action.

Step 3.3 — Summary Screen

- Accordion list of topics per source. Tap to expand summary below the topic header.
- Topic difficulty shown as colored badge: Easy (green), Medium (amber), Hard (red).
- Long-press on summary text shows native share sheet to copy or share the summary.
- Sticky header shows source document name. Scroll position preserved on back navigation.
- Bottom sheet (React Native Gesture Handler) slides up with full topic details on tap.
- Floating action button launches the quiz for the currently selected source.

Step 3.4 — Flashcard Screen

- Full-screen card with 3D flip animation on tap. Uses React Native Reanimated rotateY interpolation.
- Swipe right = 'Know it' (green glow), swipe left = 'Review again' (red glow). Haptic on each.

- Progress dots at the top show current card position in the deck.
- Bottom action row: thumbs up, thumbs down, skip, hint (reveals hint text).
- Session summary modal at end of deck: cards mastered, cards to review, time taken.
- Offline mode: cards cached in AsyncStorage. Study without internet, sync results on reconnect.

Step 3.5 — Quiz Screen

- Full-screen question card. Four option buttons below with large tap targets (min 48dp height).
- After answering: correct option turns green, wrong selection turns red. Cannot change answer.
- Explanation text slides in from bottom after answering. 'Next' button proceeds to next question.
- Progress bar at top: current question / total questions.
- Results screen: score ring animation, topic breakdown list, option to retry or share score.
- Share result: generates a shareable score card image using Expo Sharing.

Step 3.6 — Audio Player Screen

- Full-screen player with topic art (generated gradient), title, and waveform visualization.
- Play/pause, previous/next topic, seek slider, playback speed picker (0.75x–2x).
- Background audio continues when app is backgrounded. Lock screen controls via Expo AV.
- Transcript panel: scrollable below the player. Tap any paragraph to seek audio to that point.
- Download button caches MP3 to device storage via Expo FileSystem for offline playback.
- Mini player bar appears at the bottom of all tab screens when audio is playing.

Step 3.7 — Study Planner Screen

- Horizontal scroll week bar at top (Mon–Sun). Active day highlighted. Tap to jump to that day.
- Day detail: scrollable list of study sessions (topic, duration, type label: Learn / Review / Quiz).
- Tap a session to mark complete — animated checkmark, green background on the card.
- Missed session banner: sticky alert bar with days overdue count and Catch Up button.
- Exam countdown widget: days remaining, subject name, readiness score.
- Bottom sheet to add a new exam: subject, exam date, daily hours picker.

Step 3.8 — Dashboard Screen

- Pull-to-refresh updates all dashboard metrics.
- Readiness ring (Victory Native): animated fill on load. Color: red < 50%, amber 50–75%, green 75%+.
- Topic radar chart: same 6-axis design as web, optimized for 375px screen width.
- Study streak card: fire emoji placeholder, streak count, best streak, total hours.
- Weak topics chips: tap any chip navigates directly to that topic's flashcard deck.
- Recent activity list: scrollable timeline of last 10 study events with relative timestamps.

Step 3.9 — Profile & Settings Screen

- User avatar (initials-based), name, email, member since date.
- Notification settings: toggle daily reminder, set reminder time via native time picker.
- Theme toggle: Light / Dark / System default (respects device setting via useColorScheme).

- Danger zone: Delete account, Clear all offline data buttons with confirmation dialogs.
- App version, terms, privacy policy links. Logout button at the bottom.

4. Push Notifications & Reminders

Step 4.1 — Expo Notification Setup

- expo-notifications handles both local (scheduled) and push (server-sent) notifications.
- On first app launch: request notification permission using Notifications.requestPermissionsAsync().
- If granted: register device with Expo Push Service, save ExpoPushToken to backend (PATCH /user/pushToken).
- Node.js backend uses @expo/server-sdk to send push messages: study reminders, plan alerts.

Step 4.2 — Local Scheduled Reminders

- User sets a daily reminder time in Settings. App schedules a daily repeating local notification.
- Content: 'Time to study! You have [N] sessions planned for today.' with topic name.
- Missed session notification: if user hasn't opened the app by 8 PM, trigger 'You missed today's plan!'.
- Exam countdown: 7-day, 3-day, 1-day reminders automatically scheduled when exam is added.

5. Offline Mode & Data Caching

Data	Storage	Strategy
Flashcards	AsyncStorage	Cached on first load. Study sessions saved locally, synced on reconnect.
Study Plan	AsyncStorage	Current week's plan persisted. Mark sessions complete offline.
Audio Files	Expo FileSystem	Downloaded MP3s stored in app's document directory.
Summaries	React Query cache	Last-fetched summaries available offline automatically.
Quiz Results	AsyncStorage queue	Offline results queued, bulk-uploaded when network returns.

Network status is monitored via NetInfo (expo-network). When offline, a subtle banner appears at the top. All write operations are queued using a simple offline queue in Zustand and flushed automatically when connectivity is restored.

6. EAS Build & App Store Submission

Step 6.1 — EAS Build Setup

- Install EAS CLI: `npm install -g eas-cli`. Log in: `eas login`.
- Initialize: `eas build:configure` — generates `eas.json` with preview and production profiles.
- Preview profile: builds unsigned APK for internal testing (Android) or .ipa for TestFlight (iOS).
- Production profile: builds signed AAB (Android) and signed IPA (iOS) for store submission.

Step 6.2 — Android Build & Play Store

- Generate upload keystore: eas credentials — EAS manages it securely in the cloud.
- Build: `eas build --platform android --profile production` → produces signed .aab file.
- Google Play Console: create app listing, upload AAB to Internal Testing track first.
- Screenshots required: phone (min 2), tablet optional. App icon: 512x512 PNG.
- Content rating questionnaire, privacy policy URL required. Review takes 1–3 days.

Step 6.3 — iOS Build & App Store

- Apple Developer account required (\$99/year). Create App ID in Apple Developer portal.
- Build: `eas build --platform ios --profile production` → EAS handles certificates and provisioning.
- Upload to App Store Connect via `eas submit --platform ios`.
- TestFlight internal testing: add testers by Apple ID, they install via TestFlight app.
- App Store review: fill in metadata (description, keywords, support URL), submit for review.
- Review takes 24–72 hours. Common rejection reasons: missing privacy policy, broken links.

Step 6.4 — OTA Updates (Over The Air)

- expo-updates allows JS bundle updates without going through app store review.
- `eas update --channel production` pushes new JS to all users within 24 hours.
- Use for: bug fixes, UI tweaks, new feature flags. Cannot change native modules via OTA.
- users see new version on next app restart. Zero downtime, no user action required.

7. Performance Optimizations

Technique	Implementation Detail
FlatList virtualization	Use FlatList (not ScrollView + map) for all lists. Renders only visible items.
Image optimization	expo-image over Image. Caches aggressively, supports blurhash placeholders.
Memo & callbacks	React.memo on expensive components. useCallback on event handlers in lists.
Lazy loading tabs	Expo Router lazy-loads tab screens — only renders when first visited.
Reanimated on UI thread	All swipe/flip animations run on the UI thread — never drops frames.

Technique	Implementation Detail
Bundle size	Use expo-modules only (no bare React Native). Tree-shake unused Expo packages.